Adaptive Model Selection for LLMs Using Contextual Bandits

MSML635 Interim Report — Group 13

Amey Hengle Sawyer Byrd Sujeet Madihalli

University of Maryland, College Park

{ameyhen, sbyrd1, sujeet}@umd.edu

Code: https://github.com/AmeyHengle/MSML635-Project

Abstract

We study LLM routing as a contextual bandit problem, where the router observes only the outcome of the model it chose and the environment keeps changing. Since the proposal we have built the full pipeline—a RouterBench stream simulator with bandit feedback, four scripted shift scenarios, four bandit policies, six baselines including RouterBench's Zero Router, and three off-policy estimators—and obtained first results on all three research questions. Three findings stand out. Linear bandits reach **96% of a full-information supervised router's reward while observing one label per query instead of eleven**, but clear the Zero Router by only 1.8%, and ϵ-greedy fails to clear it at all. Of our four shifts, only the query-mix shift materially harms the bandits, and the *static* supervised router proves more robust than they are—the opposite of our hypothesis. Finally, IPS error grows $3.9\times$ as the logging policy becomes greedier, while SNIPS and DR remain stable. We also report a blocking dependency problem with the Open Bandit Pipeline and a miscalibration of λ that made two of our shift scenarios no-ops.

1 Problem and Research Questions

Large language models vary widely in capability and price, and most queries do not need the most expensive model. A *router* decides, per query, which model answers. Most academic routers are trained with full-information labels—obtained by running every model on every query—and evaluated on static benchmarks (Hu et al., 2024; Lu et al., 2026). In deployment neither assumption holds: a router observes only the outcome of the model it chose, and models, prices, and traffic keep changing.

At round t a query with features $x_t \in \mathbb{R}^d$ arrives, the router selects $a_t \in \mathcal{A}$, and observes the reward of that arm only:

$$r(x, a) = \text{quality}(x, a) - \lambda \cdot \text{cost}(x, a), \tag{1}$$

with regret measured against the per-query oracle $a_t^\star = \arg\max_a r(x_t, a)$.

RQ1. In a stationary setting, how close do bandit routers come to a supervised router trained with full information?

RQ2. Under controlled shifts, do bandit routers adapt faster and accumulate less regret than a static supervised router?

RQ3. Can standard OPE estimators predict a routing policy's value from logs, checked against simulator ground truth?

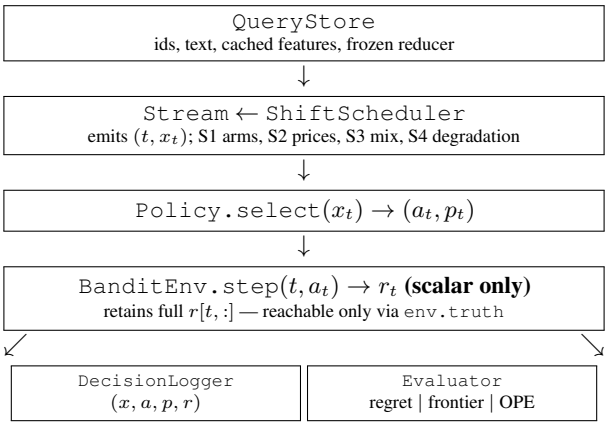


Figure 1: Implemented architecture. One arrow carries a scalar to the policy; the full reward vector reaches only the evaluator.

2 Notional Architecture

Figure 1 shows the implemented system. Two design decisions dominate it.

The feedback restriction is a type boundary, not a convention. `BanditEnv.step()` returns a single scalar. The full per-arm reward vector exists—the oracle, regret, and OPE ground truth all need it—but is reachable only through `env.truth`, a separate object handed to the evaluator and never to a policy. The most damaging available bug in this project is a policy that accidentally sees rewards it should not, and such a bug produces beautiful results; making it a structural impossibility was worth the indirection.

λ is applied downstream of the stream. The stream emits `(quality_vec, cost_vec)` separately rather than a scalar reward, so a single replay serves an entire λ sweep instead of re-streaming per point on the cost–quality frontier.

3 Progress So Far

Dataset layer (complete). We load the RouterBench 0-shot release (Hu et al., 2024): 36,497 queries $\times$ 11 models, stored wide with per-model quality, dollar cost, and response columns. The loader coerces the quality columns (shipped as `object` dtype), maps the 86 distinct `eval_name` values onto seven benchmark families, excludes MT-Bench (0.4% of rows, too small to split—the same reason RouterBench's authors excluded it), and keeps the undocumented Chinese-language tail *tagged* rather than dropped. 34 tests pass.

Model	Quality	Cost (USD)	×cheap	Oracle %
mistral-7b	0.302	4.47e-05	1.0	31.8
wizardlm-13b	0.422	7.15e-05	1.6	22.7
mixtral-8x7b	0.542	1.32e-04	2.9	16.9
codellama-34b	0.196	1.68e-04	3.7	1.2
yi-34b	0.639	1.82e-04	4.1	14.5
llama2-70b	0.331	1.98e-04	4.4	2.0
claude-instant	0.601	2.27e-04	5.1	5.0
gpt-3.5-turbo	0.619	2.38e-04	5.3	3.1
claude-v1	0.628	2.08e-03	46.5	0.5
claude-v2	0.637	2.36e-03	52.7	0.7
gpt-4	0.775	3.19e-03	71.4	1.6

Table 1: The eleven-model pool, ordered by mean per-query cost ($n = 4{,}982$). Oracle % is the share of queries the per-query oracle assigns to that arm at $\lambda = 10$. Cost spans 72×; quality does not follow it.

Check	Evidence	Status
(i) reward/regret units	unit tests, λ monotone	pass
(ii) oracle correctness	100% vs shipped column	pass
(ii) nothing beats oracle	min margin $\geq -10^{-12}$	pass
(iii) LinUCB sublinear	2nd/1st-half ratio 0.197	pass
(iv) IPS unbiased	bias -0.019, RMSE 0.026	pass
(v) splits disjoint	34 dataset tests	pass
(v) frontier vs RouterBench	qualitative only	partial

Table 2: Verification status. (iii) and (iv) run on the synthetic stream with known parameters, which RouterBench cannot provide.

Characterising the pool. Table 1 and Figure 2 establish that this is a genuine routing problem. Cost spans 72× from `mistral-7b` to `gpt-4`, and quality does not follow cost monotonically: `codellama-34b` costs 3.8× `mistral-7b` and scores worse in aggregate. The per-family view is starker—`codellama-34b` scores 0.004 on MMLU and 0.463 on MBPP, and `llama2-70b` 0.028 on MMLU against 0.545 on HellaSwag. Mean per-model rank standard deviation across families is 1.36, so model *rankings* reorder by domain. Inter-model quality correlation averages 0.249. If orderings were stable, a context-free policy would suffice.

System layer (complete). Bandit environment with arm registry, time-varying cost model and shift scheduler; four policies (ϵ-greedy, LinUCB (Li et al., 2010), linear Thompson sampling (Agrawal & Goyal, 2013), discounted LinUCB (Russac et al., 2019)); six baselines; IPS, SNIPS and doubly robust estimators (Dudík et al., 2011); replay runner with propensity logging.

Verification. Table 2 reports the five checks from the proposal. The one we consider strongest is (ii): our recomputed oracle agrees with the `oracle_model_to_route_to` column RouterBench ships at 100% on every answerable query, so the reward bookkeeping is validated against the benchmark's authors rather than against ourselves.

4 Issues Faced

Solved: OBP is unusable on the critical path. Our proposal named the Open Bandit Pipeline (Saito et al., 2021) as our primary base. It declares `requires-python >=3.8,<3.11`; on Python 3.13 `pip` silently resolves to the much older `obp 0.4.1` rather than erroring, and when it does resolve it pulls PyTorch plus the full CUDA toolchain—several gigabytes—for estimators that are closed-form expressions of roughly thirty lines. We implemented IPS, SNIPS and DR directly, removing a hard Python ceiling and a 4 GB dependency. OBP is retained only in a separate pinned Python 3.10 environment to cross-check our implementations.

Solved: λ had no calibrated scale. Equation 1 as written in the proposal left λ dimensionless. With cost normalised by the maximum per-query cost, the trade-off is *inert* for $\lambda \lesssim 2$: the oracle's arm distribution is essentially unchanged across $\lambda \in [0.1, 5]$. The frontier bends over $\lambda \in [2, 30]$ (Figure 3); at $\lambda = 30$ the oracle spends 52% of its λ=0 cost while retaining 97.6% of its quality. Our experiment grid is therefore $\{0, 1, 2, 5, 10, 15, 20, 30, 50\}$, not $[0, 1]$. At $\lambda = 0$ the oracle is degenerate: six arms tie on average.

Solved: our first shift design produced four no-ops. We initially targeted S1/S2/S4 at GPT-4 and the cheapest arm. At $\lambda = 10$ the oracle selects neither often, so every scenario left all policies statistically indistinguishable. A shift only tests adaptation if it changes the arm a good policy would have chosen; the scenarios are now aimed at the oracle's workhorse arm. S1 and S3 now bite (Section 6.2); S2 and S4 still do not.

Solved: LinUCB cannot generate valid logs. LinUCB is a deterministic $\arg\max$, so its propensity is identically 1 and IPS degenerates. RQ3 therefore requires its own logging runs with a stochastic policy; it cannot reuse RQ1/RQ2 logs. `DecisionLog.check_loggable()` now raises rather than silently returning nonsense.

Corrected assumption: rewards are not binary. We expected quality to be $0/1$ and flagged the mismatch with LinUCB's linear-Gaussian assumption as a risk. Measured, 20.6% of (query, model) cells take fractional values (predominantly 0.25, 0.5, 0.75). The concern was overstated.

Unsolved: recovery time does not discriminate. Our proposed metric—rounds until reward returns within 5% of its pre-shift level—returns 0 for every policy in every scenario, because post-shift reward is already within tolerance on a 200-round smoothing window. The metric needs either a tighter tolerance or a per-arm formulation.

Unsolved: features and scale. Results below use TF-IDF character n-grams reduced by truncated SVD rather than the sentence-transformer embeddings of the proposal. The

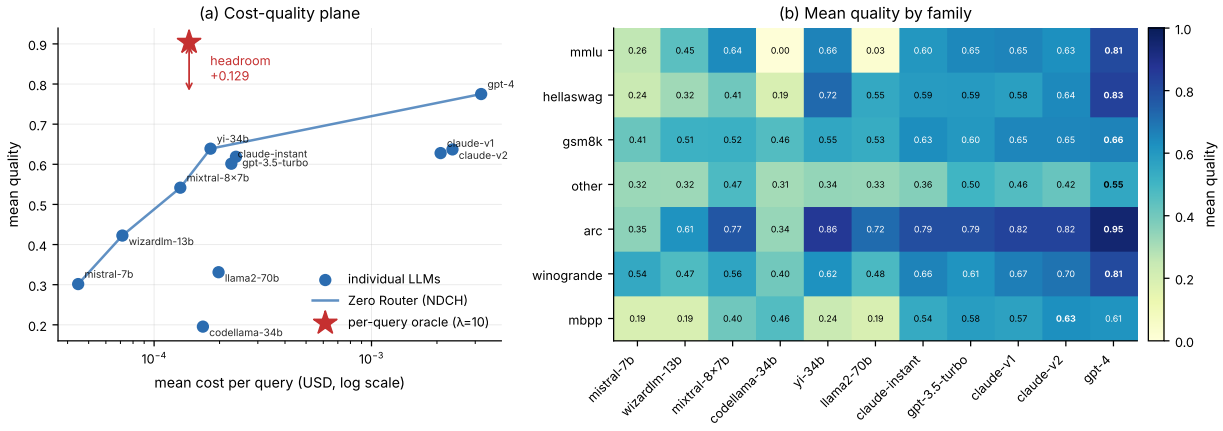


Figure 2: (a) The cost–quality plane: eleven models, the non-decreasing convex hull they span (the Zero Router, AIQ $= 0.698$, spanned by only five arms), and the per-query oracle. The vertical gap is the headroom available to a router. (b) Mean quality by family: rankings reorder across domains, which is the signal a contextual router exploits.

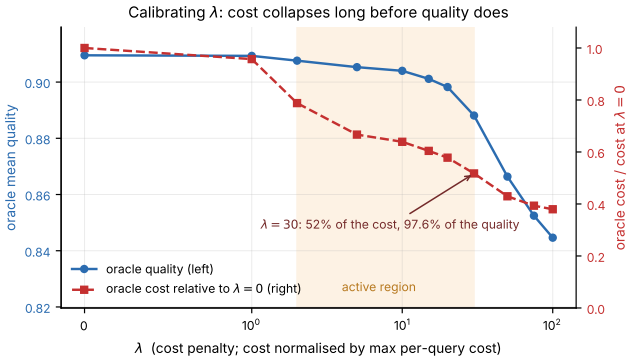


Figure 3: Calibrating λ. The cost–quality trade-off is inert below $\lambda \approx 2$ and bends over $[2, 30]$.

character-n-gram choice is defensible given the Chinese-language tail, but the comparison is not yet run. All results are on a $4{,}982$-query subsample; the loader reads the full release unchanged.

5 Baselines and State of the Art

Implemented baselines. *Per-query oracle* (upper bound); *always-cheapest* and *always-premium*; *uniform random*; *Zero Router*, RouterBench's query-agnostic probabilistic mixture over the non-decreasing convex hull; *full-information supervised router*, a per-arm ridge regression trained on calibration data with every arm's reward—the labelling regime most academic routers assume, and the reference point for RQ1; and a *confidence-based cascade* (Chen et al., 2023), evaluated directly over the full matrices since it pulls several arms per query.

The Zero Router deserves emphasis because our proposal omitted it. RouterBench's authors found their own KNN and MLP routers failed to beat it consistently, so it, not the trivial fixed policies, is the bar. On our data it is spanned by only five of eleven arms and attains AIQ $= 0.698$.

State of the art. The closest work frames routing as a contextual bandit: PILOT (Panda et al., 2025) is a LinUCB extension with a preference prior and an online multi-choice knapsack cost policy; BaRP (Wei et al., 2025) trains a preference-conditioned policy with policy gradients; ParetoBandit (Taberner-Miller, 2026) adds a primal–dual budget pacer and geometric forgetting for non-stationary serving. We note that our "discounted LinUCB" is the same forgetting mechanism ParetoBandit uses, and we claim it as a baseline rather than a contribution. Full reimplementation of PILOT and BaRP is scoped for the final report; we currently compare against RouteLLM's metrics (Ong et al., 2025) rather than its router, because its routers are binary strong/weak and our pool has eleven arms.

6 Prototype Results

All results: five seeds, $4{,}982$ queries, $3{,}486$-round streams, d=32 features plus bias, λ=10 unless stated.

6.1 RQ1 — Stationary

Table 3 and Figure 4. Linear bandits reach 95.7% (LinUCB) and 96.4% (LinTS) of the full-information supervised router's reward while observing one label per query instead of eleven. That is the central RQ1 result and it is favourable: the cost of bandit feedback is roughly four percent of reward.

The second result is less favourable. The Zero Router attains 0.5780, so LinTS clears it by only 1.8% and ϵ-greedy (0.5475) does not clear it at all. A router that cannot beat a query-agnostic probabilistic mixture has not demonstrated contextual routing, and this reproduces RouterBench's own finding for KNN and MLP routers. Every learner also over-escalates: escalation rates run 0.68–0.76 against the oracle.

Policy	Reward	% Sup.	Quality	Cost ($)	Regret	Oracle %
Per-query oracle	0.8590 ± 0.0000	140.8	0.9017	1.49e-04	0	100.0
Supervised (full info)	0.6101 ± 0.0000	100.0	0.6785	2.39e-04	868	14.9
LinTS	0.5881 ± 0.0046	96.4	0.6460	2.03e-04	944	14.6
LinUCB	0.5836 ± 0.0106	95.7	0.6354	1.81e-04	960	16.8
Discounted LinUCB	0.5806 ± 0.0075	95.2	0.6336	1.85e-04	970	16.4
Zero Router	0.5780 ± 0.0024	94.7	0.6361	2.03e-04	980	14.8
ϵ-greedy	0.5475 ± 0.0096	89.7	0.6200	2.54e-04	1086	15.6
Always cheapest	0.2875 ± 0.0000	47.1	0.3003	4.50e-05	1992	32.0
Random	0.2767 ± 0.0135	45.3	0.5172	8.42e-04	2030	8.8
Always premium	-0.1437 ± 0.0000	-23.6	0.7740	3.21e-03	3495	1.8

Table 3: RQ1 at $\lambda = 10$, mean over five seeds. "% Sup." is reward relative to the full-information supervised router. Linear bandits reach 96% of it from one label per query instead of eleven, but clear the Zero Router by only 1.8%, and ϵ-greedy does not clear it at all.

6.2 RQ2 — Under Shift

Table 4 and Figure 5 report post-shift reward lost relative to the stationary run. The result contradicts our hypothesis: the *static* supervised router is the most robust policy, losing 0.0033 under the query-mix shift where LinUCB loses 0.0517.

Two qualifications. First, only S1 and S3 produce measurable effects; S2 and S4 remain near-null even after retargeting, so we cannot yet say anything about price shocks or silent degradation. Second, within S1—where the oracle's workhorse arm is withheld and then released—discounted LinUCB loses 0.0123 against plain LinUCB's 0.0196, the one place where forgetting helps in the hypothesised direction. Seed variance is large relative to these effects.

6.3 RQ3 — Off-Policy Evaluation

Table 5 and Figure 6. Because the simulator retains every arm's reward, each estimator's true error is measurable rather than assumed. Under uniform logging all three estimators are close to unbiased (IPS bias -0.019). The informative result is what happens as the logging policy becomes greedier: minimum propensity falls from 0.091 to 0.009 and IPS RMSE rises from 0.026 to 0.102, a $3.9\times$ degradation, while SNIPS and DR rise far less ($0.017 \rightarrow 0.060$ and $0.017 \rightarrow 0.064$). The practical reading is that IPS alone should not be trusted to evaluate a routing policy from logs produced by an already-good router, which is precisely the deployment case.

7 Frameworks and Libraries

Python 3.12 managed by `uv` with a committed lockfile, so `uv sync && uv run pytest` reproduces the environment exactly. The core package depends only on NumPy, SciPy, scikit-learn, pandas and Matplotlib; PyTorch and `sentence-transformers` sit in an optional `embed` extra and HuggingFace clients in a `data` extra, both needed only for one-time offline jobs. This keeps the install light for collaborators who only run experiments. CI runs `ruff` and the test suite on every push. Bandit policies, the environment, and the OPE estimators are our own implementations; OBP is used only for cross-validation, as explained above.

8 Revised Plan and Risks

Remaining work, in priority order: (1) re-run everything at full scale (36,497 queries); (2) make S2 and S4 bite, or report that RouterBench cannot support them; (3) replace the recovery-time metric; (4) add the sentence-transformer feature backend and compare; (5) implement PILOT as a SoTA comparison; (6) sweep the Zero Router's target cost so it traces a full frontier rather than a point.

The principal risk is now clearer than at proposal time. Our RQ2 hypothesis is not currently supported: the static supervised router is more robust than the bandits under every shift we can make bite. If that survives full-scale replication, the honest contribution becomes a negative result—a characterisation of when adaptive routing does *not* pay—together with the simulator, the calibration methodology, and the OPE validation, all of which stand independently.

References

S. Agrawal and N. Goyal. Thompson sampling for contextual bandits with linear payoffs. *ICML*, 2013.

L. Chen, M. Zaharia, and J. Zou. FrugalGPT. *arXiv:2305.05176*, 2023.

M. Dudík, J. Langford, and L. Li. Doubly robust policy evaluation and learning. *ICML*, 2011.

Q. J. Hu et al. RouterBench: a benchmark for multi-LLM routing systems. *arXiv:2403.12031*, 2024.

L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. *WWW*, 2010.

Y. Lu et al. RouterArena: an open platform for comprehensive comparison of LLM routers. *ICLR*, 2026.

I. Ong et al. RouteLLM: learning to route LLMs with preference data. *ICLR*, 2025.

P. Panda, R. Magazine, C. Devaguptapu, S. Takemori, and V. Sharma. Adaptive LLM routing under budget constraints. *Findings of EMNLP*, 2025.

Policy	Stationary	S1	S2	S3	S4
Discounted LinUCB	0.5923	+0.0123	+0.0001	+0.0455	+0.0019
LinUCB	0.5950	+0.0196	-0.0002	+0.0517	+0.0015
Supervised (full info)	0.6143	+0.0000	+0.0000	+0.0033	+0.0000
ϵ-greedy	0.5616	+0.0028	+0.0010	+0.0052	+0.0031

Table 4: RQ2: post-shift reward lost versus the stationary run (positive is worse), $\lambda = 10$, five seeds. The static supervised router is the most robust policy, contradicting our hypothesis. S2 and S4 remain near-null.

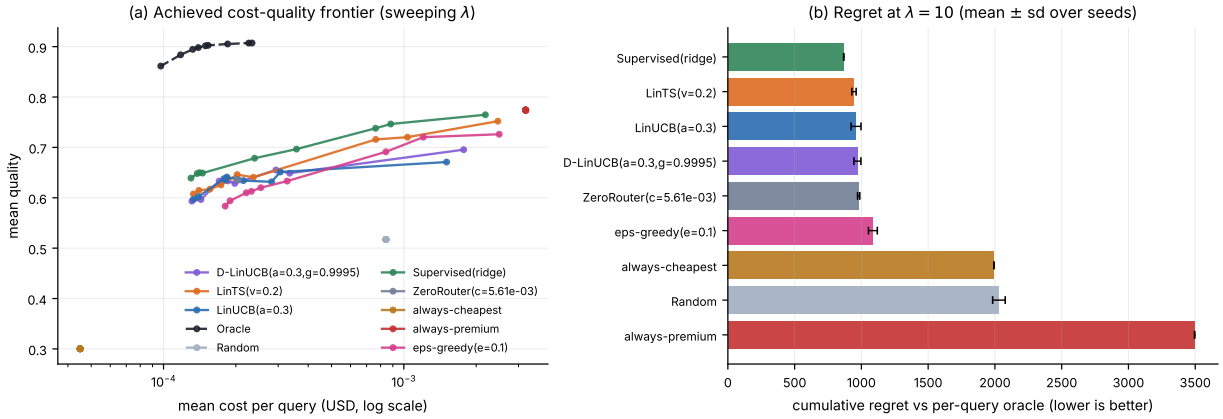


Figure 4: RQ1. (a) Achieved cost–quality frontier as λ sweeps. (b) Cumulative regret at $\lambda = 10$, mean $\pm$ sd over five seeds.

Logging policy	Est.	Bias	RMSE	min p
uniform	DR	-0.0074	0.0165	0.091
	IPS	-0.0187	0.0262	0.091
	SNIPS	-0.0061	0.0167	0.091
eps-greedy(0.3)	DR	+0.0023	0.0275	0.027
	IPS	+0.0129	0.0489	0.027
	SNIPS	-0.0038	0.0247	0.027
eps-greedy(0.1)	DR	-0.0149	0.0641	0.009
	IPS	-0.0174	0.1018	0.009
	SNIPS	-0.0014	0.0599	0.009

Table 5: RQ3: bias and RMSE against simulator ground truth. IPS degrades 3.9$\times$ as the logging policy becomes greedier; SNIPS and DR do not.

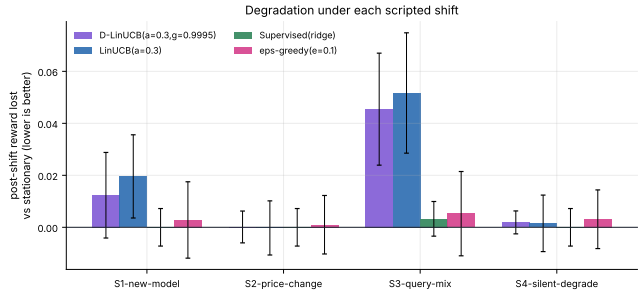


Figure 5: RQ2. Post-shift reward lost versus stationary. Only S1 and S3 produce measurable effects; the supervised router is the most robust.

Y. Russac, C. Vernade, and O. Cappé. Weighted linear bandits for non-stationary environments. *NeurIPS*, 2019.

Y. Saito, S. Aihara, M. Matsutani, and Y. Narita. Open bandit dataset and pipeline. *NeurIPS D&B*, 2021.

A. Taberner-Miller. ParetoBandit: budget-paced adaptive routing for non-stationary LLM serving. *arXiv:2604.00136*, 2026.

W. Wei et al. Learning to route LLMs from bandit feedback: one policy, many trade-offs. *arXiv:2510.07429*, 2025.

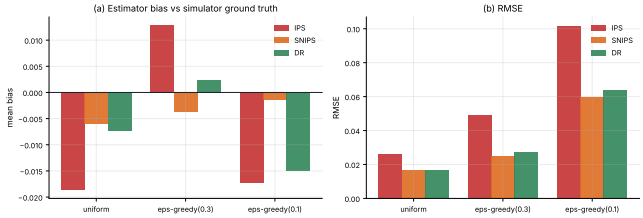


Figure 6: OPE accuracy. IPS degrades sharply as logging propensities shrink; SNIPS and DR remain stable.